COMP364 Quiz 4

Question 1. Computing convolution output values

Each of the following questions describes two adjacent layers in a one-dimensional convolutional neural network: upstream layer A and downstream layer B. In each case, give the values computed at B based on inputs from A. Figure **??** summarizes the relevant channel values and kernels used in each part of the question.

$$
\begin{aligned}
A_0 &= (5, 0, -1, 1, 0) \\
k_{00} &= (1, -2, 1) \\
b_0 &= 2 \\
s &= 1
\end{aligned}
$$

$$
\begin{aligned}
A_0 &= (5, 0, -1, 1, 0) \\
A_1 &= (0, -2, 4, 0, 1) \\
A_2 &= (6, 4, 3, -3, 0) \\
k_{00} &= (1, -2, 1) \\
k_{10} &= (4, 0, -3) \\
k_{20} &= (0, 1, 2) \\
b_0 &= 2 \\
k_{01} &= (3, 3, 3) \\
k_{11} &= (-1, 0, -1) \\
k_{21} &= (2, 1, 0) \\
b_1 &= 0 \\
s &= 2
\end{aligned}
$$

Figure 1: **Top:** Channel value and kernel for question **??**(a). **Bottom:** Channel values and kernels for questions **??**(b)–(e).

(a) **(5 points)** Layer A has one channel, A_0, of length 5. Its output is $A_0 = (5, 0, -1, 1, 0)$. B has 1 channel, B_0. Channel B_0 is produced from channel A_0 by cross-correlating with a kernel of size 3; the kernel is $k_{00} = (1, -2, 1)$. Channel B_0 has bias $b_0 = 2$. Layer B employs a stride of

$s = 1$. More generally, the notation k_{ij} denotes the kernel mapping channel A_i to channel B_j. The symbol b_i designates the bias of channel B_i, and s designates the stride for layer B.[1]

Solution.

The cross-correlation values are

$$1 \cdot 5 + (-2) \cdot 0 + 1 \cdot (-1) + 2 = 6$$
$$1 \cdot 0 + (-2) \cdot (-1) + 1 \cdot 1 + 2 = 5$$
$$1 \cdot (-1) + (-2) \cdot 1 + 1 \cdot 0 + 2 = -1$$

So the values computed by B from A are given by $B_0 = (6, 5, -1)$.

(b) **(5 points)** Everything is identical to the previous question except that the kernel stride is $s = 2$.

Solution.

The cross-correlation values are

$$1 \cdot 5 + (-2) \cdot 0 + 1 \cdot (-1) = 4$$
$$1 \cdot (-1) + (-2) \cdot 1 + 1 \cdot 0 = -3$$

Adding the bias gives the values computed by B from A as $B_0 = (6, -1)$.

(c) **(5 points)** Everything is identical to (b) except that A now has a second channel with outputs $A_1 = (0, -2, 4, 0, 1)$, and B has a corresponding new kernel $k_{10} = (4, 0, -3)$.

Solution.

The cross-correlation values for channel A_0 to B_0 are the same as before: $(4, -3)$.

The cross-correlation values for channel A_1 to B_0 are

$$4 \cdot 0 + 0 \cdot (-2) + (-3) \cdot 4 = -12$$
$$4 \cdot 4 + 0 \cdot 0 + (-3) \cdot 1 = 13$$

Summing the two channels gives

$$4 + (-12) = -8$$
$$(-3) + 13 = 10$$

Adding the bias gives the values computed by B from A are $B_0 = (-6, 12)$.

(d) **(5 points)** The same as (c) except now we add a third channel to A: this is channel A_2 with outputs $A_2 = (6, 4, 3, -3, 0)$ and kernel $k_{20} = (0, 1, 2)$.

[1] In the engineering literature, the word *convolution* refers to an operation in which the kernel is flipped. The correct technical term for applying a kernel without flipping is *cross-correlation*, not convolution. However, it is common in the CNN literature to use cross-correlation and convolution interchangeably, and the kernels are never flipped. We use this standard CNN convention here, so kernels are not flipped.

Solution.

As before, channels A_0 and A_1 output $(4, -3)$ and $(-12, 13)$ respectively. The cross-correlation values for channel A_2 are

$$0 \cdot 6 + 1 \cdot 4 + 2 \cdot 3 = 10$$
$$0 \cdot 3 + 1 \cdot (-3) + 2 \cdot 0 = -3$$

Summing the three channels gives

$$4 + (-12) + 10 = 2$$
$$(-3) + 13 + (-3) = 7$$

Adding the bias, the values computed by B from A are $B_0 = (4, 9)$.

(e) **(10 points)** The same as (d) except now we add a second channel to B, so A has three channels with IDs 0, 1, 2; and B has two channels with IDs 0, 1. The bias for channel B_1 is $b_1 = 0$, and the new kernels for channel B_1 are

$$k_{01} = (3, 3, 3)$$
$$k_{11} = (-1, 0, -1)$$
$$k_{21} = (2, 1, 0)$$

Solution.

As before, values computed by B_0 from A are $B_0 = (4, 9)$. For the new channel B_1, the three new kernels give:

$$\begin{aligned}
k_{01} :& 3 \cdot 5 + 3 \cdot 0 + 3 \cdot (-1) = 12, \text{then next stride…} \\
& 3 \cdot (-1) + 3 \cdot 1 + 3 \cdot 0 = 0 \\
& \to (12, 0) \\
k_{11} :& -1 \cdot 0 + 0 \cdot (-2) + (-1) \cdot 4 = -4, \text{then next stride…} \\
& -1 \cdot 4 + 0 \cdot 0 + (-1) \cdot 1 = -5 \\
& \to (-4, -5) \\
k_{21} :& 2 \cdot 6 + 1 \cdot 4 + 0 \cdot 3 = 16, \text{then next stride…} \\
& 2 \cdot 3 + 1 \cdot -3 + 0 \cdot 0 = 3 \\
& \to (16, 3)
\end{aligned}$$

Summing the three channels gives

$$12 + (-4) + 16 = 24$$
$$0 + (-5) + 3 = -2$$

So $B_1 = (24, -2)$.

Final answer: $B_0 = (4, 9)$, $B_1 = (24, -2)$.

Question 2. Computing convolution output sizes (33 points)

	channels in A	length of A	kernel width	kernel stride	padding (at each end)	bias	channels in B	length of B	num parameters in B	num ignored inputs
(a)	1	64	8	1	0	False	1			
(b)						True				
(c)	6									
(d)							10			
(e)						False				
(f)				2						
(g)			7	1						
(h)				2						
(i)					1					
(j)					2					
(k)					3					

Figure 2: Details for question **??**. Fill in the last 3 columns.

Figure **??** on page **??** describes various configurations for two consecutive layers, A and B, in a one-dimensional convnet. Each row of the table shows only the *changes* in configuration values compared to the previous row. So you can assume that all other configuration values remain the same as the previous row. For example, configuration (b) is the same as (a) except that (b) employs biases in its units. The changes are cumulative. For example, (c) is the same as (b) except that (c) has 6 channels, and (c) *does* employ biases, as this property is inherited from (b).

The task for this question is to fill in the last three columns of figure **??** on page **??**.

Solution.

The solution is summarized in figure **??** on page **??**. Detailed explanations are given here.

(a) There are two possible approaches for the output shape: Either (i) derive the answer from first principles, or (ii) use a memorized formula. For (i), note A has indices 0–63, and the width-8 kernel with stride 1 covers windows 0–7, 1–8, 2–9, …, 56–63. So there are 57 outputs. For (ii), use $L' = \lfloor (L - w)/s \rfloor + 1$, where L' is output length, L is input length, w is kernel width, and s is stride. This gives $L' = \lfloor (64 - 8)/1 \rfloor + 1 = 57$, as before. The parameter count comprises 8 weights for the kernel, for a total of 8 parameters. There are zero ignored inputs because the final kernel window is 56–63, which includes the final input.

(b) The bias adds one parameter for $B's$ single channel, so the only change to the answer is that there are now 9 parameters.

(c) The output shape remains the same even with the additional channels from A, but A's 6 channels mean there are now $6 \cdot 8 = 48$ kernel parameters, for a total of 49 including the bias.

(d) With 10 channels in B, there are now $6 \cdot 10 \cdot 8 = 480$ kernel parameters and 10 biases, for a total of 490 parameters.

(e) Without the biases, there are 480 parameters.

	channels in A	length of A	kernel width	kernel stride	padding (at each end	bias	channels in B	length of B	num parameters in B	num ignored inputs
(a)	1	64	8	1	0	False	1	57	8	0
(b)						True			9	
(c)	6								49	
(d)							10		490	
(e)						False			480	
(f)				2				29		
(g)			7	1				58	420	
(h)				2				29		1
(i)					1			30		0
(j)					2			31		
(k)					3			32		

Figure 3: Solution to question **??**

(f) We compute the output length again from first principles and from the formula. With a stride of 2, the kernel windows are 0–7, 2–9, 4-11, …, 56–63; this gives $56/2 + 1 = 29$ outputs. The formula gives $L' = \lfloor (L - w)/s \rfloor + 1 = (64 - 8)//2 + 1 = 29$. There are no ignored inputs because the final window includes 63.

(g) Kernel width 7 with stride 1 gives windows 0–6, 1–7, 2–8, …, 57–63; so there are 58 outputs. The formula gives $L' = \lfloor (L - w)/s \rfloor + 1 = (64 - 7)//1 + 1 = 58$. There are no ignored inputs because the final window includes 63.

(h) Switching to stride 2 gives windows 0–6, 2–8, 4–10, …, 56–62; so there are $56/2+1 = 29$ outputs. The final window misses the input with index 63, so there is one ignored input. (Additional explanation: Recall that PyTorch ignores inputs that do not fit into a full kernel window.)

(i) With padding of 1 at each end, we get additional indices -1 and 64. So the windows are now -1–5, 1–7, 3–9, …, 57–63. Some careful counting shows that there are 30 windows here. The formula including padding p is $L' = \lfloor (L - w + 2p)/s \rfloor + 1 = (64 - 7 + 2)//2 + 1 = 30$. The final window misses index 64, but that doesn't matter, because index 64 was created from padding. The final window does include index 63, so there are no ignored inputs.

(j) With padding of 2 at each end, we get additional indices $-2, -1$ and $64, 65$. So the windows are now -2–4, 0–6, 2–8, …, 56–62, 58–64. There are 31 windows here. The formula gives $L' = \lfloor (L - w + 2p)/s \rfloor + 1 = (64 - 7 + 4)//2 + 1 = 31$. The final window misses index 65, but that doesn't matter, because index 65 was created from padding. The final window does include index 63, so there are no ignored inputs.

(k) With padding of 3 at each end, we get additional indices $-3, -2, -1$ and $64, 65, 66$. So the windows are now -3–3, -1–5, 1–7, …, 57–63, 59–65. There are 32 windows here. The formula gives $L' = \lfloor (L - w + 2p)/s \rfloor + 1 = (64 - 7 + 6)//2 + 1 = 32$. The final window misses index 66, but that doesn't matter, because index 66 was created from padding. The final window does include index 63, so there are no ignored inputs.

```
import torch
import torch.nn as nn
loss_fn = nn.MSELoss()
target = torch.tensor([[1, 2], [20, 21]], dtype=torch.float32)
input = torch.tensor([[1.1, 2], [20, 20]], dtype=torch.float32, requires_grad=True)
num_vals = input.numel()
print(f"num_vals: {num_vals}")
loss = loss_fn(input, target)
loss_val = loss.item()
print(f"loss_val*num_vals: {loss_val * num_vals}")
print(f"loss_val: {loss_val}")
loss.backward()
print(f"dLoss/dInput * num_vals: \n{input.grad * num_vals}")
print(f"dLoss/dInput: \n{input.grad}")
optimizer = torch.optim.SGD([input], lr=0.1)
optimizer.step()
print(f"Updated input: \n{input.detach()}")
```

Figure 4: PyTorch code for question **??**

Question 3. Computing convolution output size with additional layer (10 points)

Consider a convolutional network that has layers A and B exactly as in the final part of the previous question, **??**(k). We add another layer C, which has 5 channels and employs a kernel of width 7, stride 2, no bias, and padding 3 on each side. Give the (a) shape and (b) parameter count of layer C.

Solution.

(a) Using $L' = \lfloor (L - w + 2p)/s \rfloor + 1$ output length is $(32 - 7 + 6)//2 + 1 = 16$. Output shape is thus $(5, 16)$

(b) There are 10 input channels, 5 output channels, and 7 kernel weights. So parameter count is $10 \cdot 5 \cdot 7 = 350$.

Question 4. Regression using MSE and PyTorch

(a) **(2 points)** Given output values $x_1, x_2, \ldots, x_n$ and target values $t_1, t_2, \ldots, t_n$, what function is minimized when using mean squared error (MSE)?

Solution.

$\sum_{i=1}^{n} (x_i - t_i)^2$

(b) **(15 points)** Figure **??** shows some code for performing regression in PyTorch using mean squared error (MSE), and figure **??** shows the output of this code. Give a brief explanation of each line of output.

```
1  num_vals: 4
2  loss_val*num_vals: 1.0099999904632568
3  loss_val: 0.2524999976158142
4  dLoss/dInput * num_vals:
5  tensor([[ 0.2000,  0.0000],
6          [ 0.0000, -2.0000]])
7  dLoss/dInput:
8  tensor([[ 0.0500,  0.0000],
9          [ 0.0000, -0.5000]])
10 Updated input:
11 tensor([[ 1.0950,  2.0000],
12         [20.0000, 20.0500]])
```

Figure 5: Output of code in figure **??**

Solution.

- Line 1: The input is a 2x2 tensor with 4 elements.
- Line 2: The MSE here is $(1.1-1)^2+(22-21)^2=1.01$.
- Line 3: By default PyTorch divides the loss by the number of elements, so `loss_val` is $1.01/4=0.2525$.
- Lines 4–6: The derivative of $(x_i-t_i)^2$ with respect to x_i is $2(x_i-t_i)$. Top left entry is thus $2\times(1.1-1)=0.2$, bottom right entry is $2\times(20-21)=-2$.
- Lines 7–9: Again, Pytorch divides by the number of elements, so the previous values are divided by 4.
- Lines 6–8: SGD update is $-\lambda dL/dx$, where λ is the learning rate, L is the loss, and x is an input or parameter. Top left update is thus $-0.1\times0.05=-0.005$, yielding $1.1-0.005=1.095$. Bottom right update is $-0.1\times(-0.5)=0.05$, yielding $20+0.05=20.05$.

Total number of points: ??